:

Normalización y optimización del csv-Enciclopedia Pokémon

Proceso de normalización

1. **Primera Forma Normal (1FN):** eliminación de atributos multivaluados.

   Regla: cada columna debe contener valores atómicos y no debe haber grupos repetidos.

- **Diagnóstico de Fallos:**

  1. **Tipos:** las columnas tipo1 y tipo2 representaban una lista de atributos en una sola entidad.

  2. **Habilidades:** la celda habilidades contenía cadenas de texto con múltiples valores (ej: "Overgrow, Chlorophyll"), impidiendo consultas granulares.

- **Acciones de Optimización:**

  - **Creación de Pokemon_Tipo:** se descompusieron los tipos en registros individuales vinculados por No_Pokedex.

  - **Implementación de Habilidades y Pokemon_Habilidad:** se extrajeron las habilidades a un catálogo único, utilizando una tabla de intersección para permitir una relación de uno a muchos (1:N), cumpliendo con la atomicidad requerida.

2. **Segunda Forma Normal (2FN):** eliminación de dependencias parciales.

   Regla: Todos los atributos no clave deben depender funcionalmente de la clave primaria completa.

Diagnóstico de Fallos:

1. El archivo original mezclaba datos de identidad (Nombre) con datos de rendimiento (Stats) y datos biológicos (Tipos), generando una tabla saturada donde no todo dependía exclusivamente del ID.

Acciones de Optimización:

Fragmentación Modular: se dividió la información en dos entidades clave:

1. **Pokemon_Base:** contiene la identidad central (No_Pokedex, nombre, Probabilidad_de_captura).
2. **Estadísticas:** almacena exclusivamente los atributos numéricos (Puntos de salud, ataque, defensa, etc.), vinculados mediante una Clave Foránea (FK).

Resultado: se logró una estructura modular donde cada tabla tiene una responsabilidad única y clara.

3. Tercera Forma Normal (3FN): eliminación de dependencias transitivas

Regla: no debe haber dependencias entre atributos que no sean la clave primaria (evitar que A dependa de B y B de la PK).

Diagnóstico de Fallos:

1. **Redundancia de Texto:** el nombre del tipo (ej: "Fuego") se repetía en cientos de filas, aumentando el peso del archivo y el riesgo de errores de dedo.
2. **Matriz de Combate:** los multiplicadores de daño (contraataques) estaban asociados al Pokémon, cuando en realidad dependen del Tipo elemental.

Acciones de Optimización:

- **Catálogo de Tipos:** creación de una tabla maestra con IDs numéricos, eliminando la redundancia de texto.
- **Matriz de Tipo_Efectividad:** se extrajo la lógica de combate. Ahora, el multiplicador depende de la interacción entre id_tipo_ataque e id_tipo_defensa.

Resultado: se eliminó la dependencia transitiva; la efectividad ahora es una propiedad de los tipos, no de los Pokémon individuales.

Proceso de optimización

1. Resolución de Duplicados Críticos en Habilidades

Detección de Anomalía: se identificó que la función .split() en el proceso inicial conservó caracteres de formato de lista ([,], '), generando duplicados falsos (ej: 'Overgrow' vs Overgrow]).

Acción de Ingeniería:

- **Atomización por Regex:** se implementó una limpieza profunda mediante Expresiones Regulares para eliminar cualquier carácter no alfanumérico en los nombres de las habilidades.

- **Mapeo de Integridad:** se ejecutó un rastreo de las relaciones originales Pokémon-Habilidad para actualizar los registros a los nuevos IDs únicos, eliminando registros huérfanos y garantizando la Integridad Referencial.

Resultado Final: reducción del catálogo de habilidades en más de un 50%, eliminando la redundancia y asegurando que cada registro en Habilidades.csv sea único y estético.

2. Consistencia de Formato (Estandarización Textual)

Acción: aplicación de una transformación de capitalización uniforme (.str.capitalize()) a todos los registros en los catálogos de Tipos y Habilidades.

Justificación: en bases de datos relacionales, la consistencia en el "Case" es vital para evitar fallos en las cláusulas WHERE y JOIN. Esto garantiza la coincidencia exacta de datos (ej: "Fuego" siempre será igual a "Fuego"), eliminando discrepancias por ingresos accidentales en minúsculas.

3. Refactorización de Tipos Numéricos (Eficiencia de Memoria)

Acción: proceso de *Type Casting* forzando el tipado de datos a Entero (Int64) en la tabla de Estadísticas y Punto Flotante (Float64) en la tabla de Tipo_Efectividad.

Justificación: al procesar los CSV, los valores numéricos suelen interpretarse como cadenas de texto (Strings). Al convertirlos a tipos numéricos puros:

- **Optimización de Almacenamiento:** se reduce el peso físico de los archivos.

- **Preparación para MySQL:** se facilita la asignación de tipos eficientes como SMALLINT o DECIMAL en lugar de VARCHAR.

- **Capacidad de Análisis:** se habilitan funciones matemáticas, cálculos de promedios y ordenamiento numérico directo en el motor de base de datos.